

CS 230 Distributed Computer Systems

Homework 3

1 Six Nested Loop Cases

Here, I used a Java program to simulate the process. I choose n of 3. At first, I defined three matrices: a , b and c as follows:

```
int[] [] a = {
    {1, 2, 3},
    {3, 1, 2},
    {2, 3, 1}
};

int[] [] b = {
    {1, 1, 1},
    {2, 2, 2},
    {3, 3, 3}
};

int[] [] c = {
    {1, 2, 3},
    {1, 2, 3},
    {1, 2, 3}
};
```

Next, we can use a nested loop to print every step in the matrix manipulation. This is how we mimic the (i, j, k) case:

```
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 3; j++) {
        for (int k = 0; k < 3; k++) {
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
            System.out.println("c" + i + j + " = " + c[i][j]);
        }
    }
}
```

By changing the order of the loops, we can mimic other cases. For example, this is how we mimic the (i, k, j) case:

```
for (int i = 0; i < 3; i++) {
    for (int k = 0; k < 3; k++) {
        for (int j = 0; j < 3; j++) {
            c[i][j] = c[i][j] + a[i][k] * b[k][j];
            System.out.println("c" + i + j + " = " + c[i][j]);
        }
    }
}
```

In the output of the program, we can get the execution order of the process.

1.1 (i, j, k) Case

In this case, the execution order is as follows:

```
c00 c00 c00 c01 c01 c01 c02 c02 c02
c10 c10 c10 c11 c11 c11 c12 c12 c12
c20 c20 c20 c21 c21 c21 c22 c22 c22
```

As we can see from the order, since k is in the most inner loop, every c(i, j) computation will stay on current c(i, j) until k increased from 0 to n - 1. As the i increases from 0 to n - 1, in each i, j increases from 0 to n - 1.

1.2 (i, k, j) Case

In this case, the execution order is as follows:

```
c00 c01 c02 c00 c01 c02 c00 c01 c02
c10 c11 c12 c10 c11 c12 c10 c11 c12
c20 c21 c22 c20 c21 c22 c20 c21 c22
```

We can regard every 3 c(i, j) as one unit, inside every unit, j increases from 0 to n - 1, since j is in the most inner loop. The computation will stay on the unit until k increased from 0 to n - 1, since the k is on the outer loop of j. The i increases every 3 computation unit from 0 to n - 1.

1.3 (j, i, k) Case

In this case, the execution order is as follows:

```
c00 c00 c00 c10 c10 c10 c20 c20 c20
c01 c01 c01 c11 c11 c11 c21 c21 c21
c02 c02 c02 c12 c12 c12 c22 c22 c22
```

As we can see from the order, since k is in the most inner loop, every $c(i, j)$ computation will stay on current $c(i, j)$ until k increased from 0 to $n - 1$. As the j increases from 0 to $n - 1$, in each j , i increases from 0 to $n - 1$.

1.4 (j, k, i) Case

In this case, the execution order is as follows:

```
c00 c10 c20 c00 c10 c20 c00 c10 c20
c01 c11 c21 c01 c11 c21 c01 c11 c21
c02 c12 c22 c02 c12 c22 c02 c12 c22
```

We can regard every 3 $c(i, j)$ as one unit, inside every unit, i increases from 0 to $n - 1$, since i is in the most inner loop. The computation will stay on the unit until k increased from 0 to $n - 1$, since the k is on the outer loop of i . The j increases every 3 computation unit from 0 to $n - 1$.

1.5 (k, i, j) Case

In this case, the execution order is as follows:

```
c00 c01 c02 c10 c11 c12 c20 c21 c22
c00 c01 c02 c10 c11 c12 c20 c21 c22
c00 c01 c02 c10 c11 c12 c20 c21 c22
```

We can regard every 9 $c(i, j)$ as a computation unit. When an unit is finished, the k increments, then start a new unit, this process last until k increases from 0 to $n - 1$. In side every unit, the i increases from 0 to $n - 1$. In every i , j increases from 0 to $n - 1$.

1.6 (k, j, i) Case

In this case, the execution order is as follows:

```
c00 c10 c20 c01 c11 c21 c02 c12 c22
c00 c10 c20 c01 c11 c21 c02 c12 c22
c00 c10 c20 c01 c11 c21 c02 c12 c22
```

We can regard every 9 $c(i, j)$ as a computation unit. When an unit is finished, the k increments, then start a new unit, this process last until k increases from 0 to $n - 1$. In side every unit, the j increases from 0 to $n - 1$. In every j , i increases from 0 to $n - 1$.

2 N-Processor Ring Mapping

2.1 (i, j, k) Case

The SPMD program of this case becomes:

```
Do All P(i = 0, n - 1):
  For j = i, do n times:
    For k = 0, n - 1, do:
      c(i, j) = c(i, j) + a(i, k) + b(k, j)
    endfor
  Every Processor i sends its current
  column of B to processor (i + 1) mod (n)
  endfor
enddoall
```

Now let's see the execution order of this program again:

```
c00 c00 c00 c01 c01 c01 c02 c02 c02
c10 c10 c10 c11 c11 c11 c12 c12 c12
c20 c20 c20 c21 c21 c21 c22 c22 c22
```

We can see that the first row's i is all 0, while the second row's i and third row's i is 1 and 2. It means we can unfold the program on i . Each row is a processor. In each processor, we only need to store $a(i, 0 \dots n - 1)$, $b(i, 0 \dots n - 1)$ and $c(i, 0 \dots n - 1)$, since it's paralleled by i , all the i in the processor is the same.

```
Time =  $O(n^2)$  Multiplication  
Memory =  $O(n)$   
Communication =  $O(n^2)$  Network Cycles
```

2.2 (i, k, j) Case

The SPMD program of this case becomes:

```
Do All P(i = 0, n - 1):  
    For k = i, do n times:  
        For j = 0, n - 1, do:  
            c(i, j) = c(i, j) + a(i, k) + b(k, j)  
        endfor  
    Every Processor i sends its current  
    column of B to processor (i + 1) mod (n)  
    endfor  
enddoall
```

Now let's see the execution order of this program again:

```
c00 c01 c02 c00 c01 c02 c00 c01 c02  
c10 c11 c12 c10 c11 c12 c10 c11 c12  
c20 c21 c22 c20 c21 c22 c20 c21 c22
```

It is similar to the (i, j, k) case, since the very outer loop is also i, so it also can be paralleled by i. Each row is a processor. In each processor, the i remains unchanged in the process. The storage scheme remains the same. However, because the order of j loop and k loop is swapped, the execution order inside the processor will change too.

```
Time =  $O(n^2)$  Multiplication  
Memory =  $O(n)$   
Communication =  $O(n^2)$  Network Cycles
```

2.3 (j, i, k) Case

The SPMD program of this case becomes:

```

Do All P(j = 0, n - 1):
  For i = j, do n times:
    For k = 0, n - 1, do:
      c(i, j) = c(i, j) + a(i, k) + b(k, j)
    endfor
  Every Processor j sends its current
  column of A to processor (j + 1) mod (n)
  endfor
enddoall

```

Now let's see the execution order of this program again:

```

c00 c00 c00 c10 c10 c10 c20 c20 c20
c01 c01 c01 c11 c11 c11 c21 c21 c21
c02 c02 c02 c12 c12 c12 c22 c22 c22

```

We can see that, in each row, the j remains the same. For example, in the first row, the j s are all zero. So we can parallel the program by j . We can regard each row as a processor. In each processor, we only store data of $a(0 \dots n - 1, j)$, $b(0 \dots n - 1, j)$ and $c(0 \dots n - 1, j)$, since in every processor, j remains the same inside.

```

Time =  $O(n^2)$  Multiplication
Memory =  $O(n)$ 
Communication =  $O(n^2)$  Network Cycles

```

2.4 (j, k, i) Case

The SPMD program of this case becomes:

```

Do All P(j = 0, n - 1):
  For k = j, do n times:
    For i = 0, n - 1, do:
      c(i, j) = c(i, j) + a(i, k) + b(k, j)
    endfor
  Every Processor j sends its current
  column of A to processor (j + 1) mod (n)
  endfor
enddoall

```

Now let's see the execution order of this program again:

```
c00 c10 c20 c00 c10 c20 c00 c10 c20
c01 c11 c21 c01 c11 c21 c01 c11 c21
c02 c12 c22 c02 c12 c22 c02 c12 c22
```

This case is similar to the case (j, i, k) . In this case, we can also parallel the process by j . Each row is a processor. Inside each processor, j remains the same. The storage scheme is same with the case (j, i, k) , though the execution order inside the processor differs from the case (j, i, k) .

```
Time =  $O(n^2)$  Multiplication
Memory =  $O(n)$ 
Communication =  $O(n^2)$  Network Cycles
```

2.5 (k, i, j) Case

The SPMD program of this case becomes:

```
Do All P( $k = 0, n - 1$ ):
  For  $i = k$ , do  $n$  times:
    For  $j = 0, n - 1$ , do:
       $c(i, j) = c(i, j) + a(i, k) + b(k, j)$ 
    endfor
  Every Processor  $k$  sends its current
  column of  $A$  to processor  $(i + 1) \bmod (n)$ 
  endfor
enddoall
```

Now let's see the execution order of this program again:

```
c00 c01 c02 c10 c11 c12 c20 c21 c22
c00 c01 c02 c10 c11 c12 c20 c21 c22
c00 c01 c02 c10 c11 c12 c20 c21 c22
```

In this case, we unfold the program on k . As we can see from the order above, three rows are identical. It is because inside every processor, there is always $i - j$ nested loop. In each processor, we need to store all matrix a and matrix b . However, since the k inside one single processor is unchanged, we don't need to store the whole matrix c inside the processor.

```
Time =  $O(n^2)$  Multiplication  
Memory =  $O(n)$   
Communication = 0 Network Cycles
```

2.6 (k, j, i) Case

The SPMD program of this case becomes:

```
Do All P(k = 0, n - 1):  
  For j = k, do n times:  
    For i = 0, n - 1, do:  
      c(i, j) = c(i, j) + a(i, k) + b(k, j)  
    endfor  
    Every Processor k sends its current  
    column of B to processor (j + 1) mod (n)  
  endfor  
enddoall
```

Now let's see the execution order of this program again:

```
c00 c10 c20 c01 c11 c21 c02 c12 c22  
c00 c10 c20 c01 c11 c21 c02 c12 c22  
c00 c10 c20 c01 c11 c21 c02 c12 c22
```

In this case, we also unfold the program on k . The storage scheme is the same as case (k, i, j). However, the execution order inside the processor is different from the case (k, i, j).

```
Time =  $O(n^2)$  Multiplication  
Memory =  $O(n)$   
Communication = 0 Network Cycles
```